

Ball Balancing using a 3RRS Parallel Manipulator

Danish Nadeem 1st, William Cheung 2nd, Adam Garand 3rd, Aum Kotecha 4th

Abstract—The objective of this project is to balance a small steel ball on the centre of a 3RRS Parallel Manipulator. The manipulator this project will use will have two degrees of freedom [1]. They are the rotation relative to the x-axis (pitch), and y-axis (roll). A resistive touch sensor is used to allow the system to determine the position of the ball on the screen [2]. The system used an embedded system and a PID controller to position the ball to the centre of the platform [3]. The PID controller will orient the platform to nullify the velocity of the ball, and the embedded system will enable the system to register the position of the ball as soon as possible. The orientation of the platform will enable a sinusoidal factor of the gravitational force to counteract the velocity of the ball. Finally, the prototype is capable of centring the ball with external disturbances to the trajectory of the ball.

I. INTRODUCTION

Balancing dynamic systems in motion is a challenge in robotics and embedded systems. This requires precise motion control and a fast reaction to the position of the ball. This project, utilized and states where open source material was used. It provided an opportunity to apply course material from the Intro to Real-Time Embedded System course.

II. MOTIVATION AND BACKGROUND

This project was inspired by the Ball Balancing project created by Aaed Musa [4]. The initial motivation was to optimize the program's embedded system to utilize the Arduino Mega as the microcontroller. The Arduino Mega operates at a frequency of 16 MHz [5]. The reference project used a microcontroller called the Teensy 4.1 which operates at a frequency of 600 MHz [6]. The microcontroller in their project was overqualified for the task and would be spending most of its time idling. It was also very expensive to purchase compared to the Arduino Mega. As the project was in its assembly and testing phase, the motivation changed as the team had to apply the culmination of course material throughout the mechatronic engineering program for the prototype to centre the ball on the resistive touch screen. This project focused on the application of the course material from the Real-Time Embedded Systems course to optimize the program. It also serves as an application of the course material and how changes to the system affected the response from the control, mechanical and electrical systems.

III. CONTEXT FOR GENERAL AUDIENCE

Imagine a small steel ball resting on a moving table that tilts to keep the ball in the center. This project replicates this scenario by combining engineering disciplines: sensors measure the ball's movement, motors adjust the platform's tilt, and algorithms ensure the ball stays centred. This setup models real-world challenges like balancing loads in robotic systems

or ensuring stability during movement in precision engineering applications.

IV. SCOPE

This project focussed on the design, implementation and optimization of the embedded and control systems required to balance a steel ball using a 3RRS Parallel Manipulator. The scope includes:

Control System development

- Evaluating the performance of PI and PID controllers
- Implementation and optimization of these controllers so that they can stabilize the steel ball at the centre

Hardware Integration

- Use an Arduino microcontroller to deal with real-time computations efficiently
- Combining stepper motors, drivers, and a resistive screen into a system to balance a steel ball

Software

- Troubleshooting and optimizing code for efficient data processing like direct port manipulation
- Reduce noise from resistive screen readings for more accurate tracking of the ball position

Performance

- Analyze the entire system in its ability to accomplish the task at hand which is to balance a steel ball
- Identify areas for improvement

V. CAD

The CAD model was directly sourced from Aaed Musa's ball balancing project. The scope of the project focused on the embedded system, therefore the team did not want to allocate time and resources to redesigning a CAD model which posed no issue.

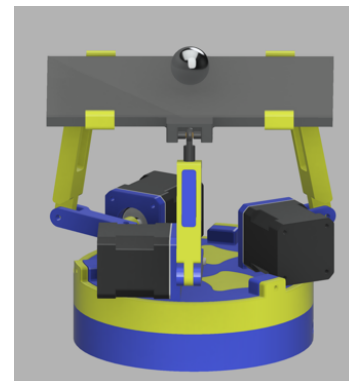


Fig. 1. CAD model from Ball Balancer Project [4]

VI. COMPONENTS

The team sourced the bill of materials from the Ball Balancer project [4]. The team made a few changes regarding the components used.

CHANGED COMPONENTS:

- Arduino Mega Microcontroller
- Nema 42-34 Ncm Stepper Motors (Bipolar)
- DRV8825 Stepper Motor Drivers
- 0.75" Steel Bearing Ball

The Arduino Mega was used instead of the Teensy 4.1 as the Mega was included in the provided kit. Smaller and more affordable stepper motors were chosen, as they were compact, cost-effective, and capable of supporting the platform's load. The steel bearing ball's smoother surface and heavier weight allowed the resistive touch screen to detect the ball more accurately.

VII. PLATFORM BALANCE

When the prototype begins, the manipulator will raise the platform and maintain an orientation parallel to the floor. When the ball is placed on the surface of the touch screen, the PID controller will calculate the error and work with the embedded system to actuate the stepper motors. The three stepper motors are the only actuators in the manipulator. The manipulator will change the orientation of the platform on which the steel ball is placed. The system utilizes the sinusoidal component of the force of gravity acting on the ball. It will be the force used to change the position of the ball [4][7]. Figure 2 shows a Free-Body Diagram of the forces acting on the ball.

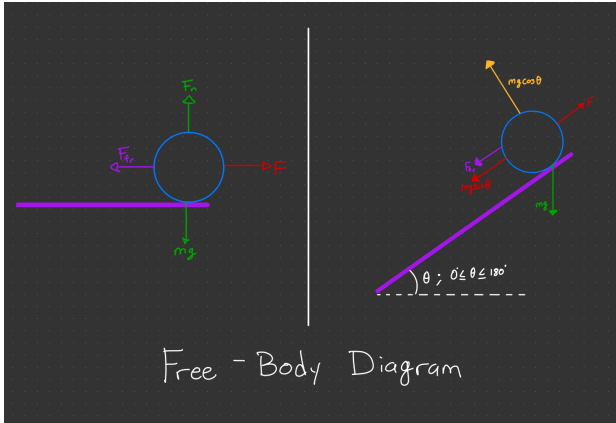


Fig. 2. Free-body diagram of two platform angles [8]

The platform must be tilted in the correct pitch and yaw angles to counteract the forces acting on the ball, slowing it down and directing it towards the center of the platform. To do this with three points of actuation, the system was thought of like this. Each servo will have a maximum and minimum position it can move, mechanically due to the physical design or in the program to limit the maximum tilt angles. This can be thought of as a vector, whose magnitude ranges from -1 to +1 and corresponds to a linear interpolation of the maximum

and minimum allowable angles. This means its magnitude maps directly to an angle through a multiplier. This allows the system to be thought of as the sum of three 2D vectors, each one rotated 120 degrees from each other - corresponding with the physical layout of the servos - with one vector directly along an axis, for example, the X axis. These three vectors control the pitch and yaw of the platform and given their fixed angular positions, only their magnitude needs to be solved. Since these vectors are made of x and y components, the output from the resistive touchpad can simply be mapped to a range from -1 to +1, and can be fed into equations that solve for these vector magnitudes. These resulting magnitudes need only be multiplied by some constant to return an angle. One vector has full influence over the roll angle, while the other two have equal and opposite influence. Meanwhile, only two vectors control the pitch angle, the third has no contribution. Finally, all three of these vectors should sum to zero, if the aim is to center the ball. Given this information, the following governing equations - summing the x components, summing the y components, and summing the magnitudes - were deduced:

$$x = ||a|| - ||b||/2 - ||c||/2$$

$$y = ||b||/2 - ||c||/2$$

$$0 = ||a|| + ||b|| + ||c||$$

This system of equations was then solved by using a technique learned in linear algebra, making use of an augmented matrix and finding the reduced row-echelon form. This led to the solutions for the vector magnitudes:

$$||a|| = x$$

$$||b|| = (-x + y)/2$$

$$||c|| = (-x - y)/2$$

These equations allow us to take an x and y input from the range -1 to +1 and turn it into three values which, through a multiplier, correspond to the angles the platform should actuate at to move the ball to such a position. All that needs to be done to provide corrective behaviour is to negate these equations or their results, which will result in the platform tilting in the complete opposite direction.

VIII. RESISTIVE TOUCHSCREEN TESTING AND TROUBLESHOOTING

For the prototype to track the ball's position and velocity a resistive touchscreen was used. It is a sensor which functions similarly to a joystick. It has two variable resistors, one for the x-axis and one for the y-axis. When pressure is applied to the screen the output voltage changes, which was used to derive a cartesian plane, as seen in Figure B. The team purchased the same model for the resistive touch screen as the open-source project used [4]. Upon assembly, the datasheet for the resistive touchscreen was used to connect with the Arduino Mega [9].

For the first test, the team used the program provided by Aaed Musa's project file [4]. The program was modified to

isolate the code to provide readings from the resistive touchscreen. The program used the library provided by Adafruit called the “Adafruit Touchscreen Library” [10].

The first test used a marble ball. An electronic calliper measured marble to have a diameter of $38.1 \pm 0.02\text{mm}$. A digital scale weighed the ball at $45.2 \pm 0.1\text{g}$.

When the ball was placed on top of the screen, the reading on the serial monitor fluctuated between readings of the position of the ball and readings when nothing was on the surface.

To test if the ball’s light weight caused the false readings a test was done by pressing one finger on the screen to observe the reading. The serial monitor displayed the same behaviour.

An attempt to resolve the issue included using different analog ports. The team tested if the ports had been damaged from previous projects. The issue persisted.

The second attempt approached the program by deriving the joystick program provided in the Arduino Library [11]. A joystick has two potentiometers, which change their voltage output and are used to derive the positions in the x and y axes [12]. The screen also uses the same technique. The connections remained the same as in the first test. The program would interpret the output voltage as they were from a joystick. When connected pressure was placed on the screen, the serial monitor displayed the maximum ADC value of 1023, being 5 volts [13]. Even with using this joystick, the signals were still showing spikes and fluctuations, leading us to believe that sensor noise or a floating state was the issue.

To try to solve the problem, two resistors were connected to the circuit. They served as pull-down resistors, under the suspicion the readings were from a floating state. One was for the circuit for the x-axis reading, and the other was for the circuit for the y-axis reading. Multiple attempts for this test were made, by changing the resistance value. It began with 100 Ω , 1 k Ω , 2 k Ω , 100 k Ω , and 1 M Ω , and their readings on the serial monitor were 932, 823, 745, 200, and 5 respectively. It did not resolve the issue as the readings remained unchanged when pressure was placed on the screen.

To further tackle this issue, a different kind of technique is to collect the x and y positions from the screen. It had the microcontroller alternate between enabling current to the x-axis and y-axis circuit one at a time. When the program needed readings from the x position, it would disable the y-axis circuit by setting the voltage input to provide no voltage. The program would take a reading, and then disable the x-axis circuit by having the microcontroller provide no voltage to the x-axis VIN pin. Enable the voltage in the pin for the y-axis circuit and get a reading. The program would repeat to get the touch screen’s readings. This technique in the program draws a theory that the noise from the previous test was due to the circuits being both active and introducing noise to the readings.

In the third approach, the team replaced the marble ball with a steel bearing ball. The decision was made after the marble was observed to wobble as it rolled. Upon inspection of the marble ball, it was noted that the surface of the ball

was not smooth and had irregularities. This caused the ball to move unpredictably. This posed an issue as the platform would struggle to predict the trajectory of the ball. The marble ball was replaced by a steel bearing ball with a diameter of $19.05 \pm 0.02\text{mm}$, and a weight of $68.0\text{g} \pm 0.1\text{g}$. The program written by Komal was designed to be portable and not optimized for the hardware [14]. The program called for many pinMode and digitalWrite commands. This resulted in a slower response time when a ball was placed on the screen. This posed another issue for the embedded system as the system was polling for the data, and calculating for a longer period. The platform would adjust for outdated data and would have a high failure rate in balancing the ball. The program was modified to implement direct port manipulation to reduce the computation time to collect the readings. This resulted in a significant increase in response time but sacrificed portability. When implementing the touchscreen, the coordinates for the screen were mapped to match the joystick readings which were used to test the stepper motors. While the direct port manipulation code was computationally faster, it still had the issue of noise, and this noise was often amplified in comparison to using a library to read the touchpad. While testing was being done, a suggestion was made to implement a detection to check whether an object was placed on the screen. The Adafruit Touch screen library was used as a reference to implement object detection. A sample program was created to test its performance. The first test had a segment from the Adafruit library copied to test its performance. It was able to detect the readings but included lots of noise in the process, and would provide false readings. The program was then modified to use direct port manipulation and added to the main program for the ball-balancing platform. In the main program, the touch detection used a moving median filter to dismiss false readings. The decision was made as the reading for an object placed on the screen fluctuated but was under the value of 10. When no object was on the screen the readings from the screen to detect pressure was 1025. The commands to read the x and y positions were nested in an if-statement with a condition being, to run the nested code if the touch screen detected pressure on the surface. The result was that the prototype struggled to keep the ball on the platform. The serial plotter of the x and y positions is shown in Figure 3. The data from Figure 3 could only occur when the screen did not detect the ball on its surface, as the global maximum and minimum readings occur when nothing is on the surface. To resolve this issue a moving median filter was added, to omit the false readings. Figure 4 shows the serial plot of the data after the filter was added.

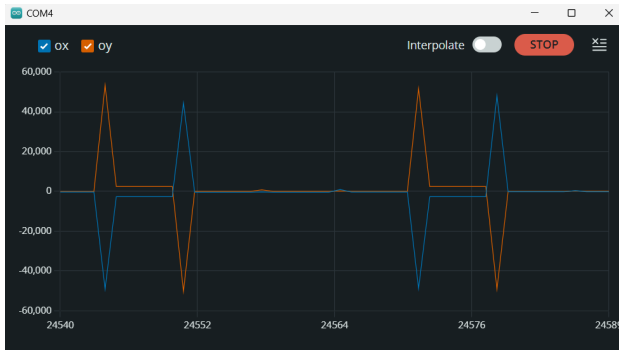


Fig. 3. Serial plot of the x and y readings after including pressure detection

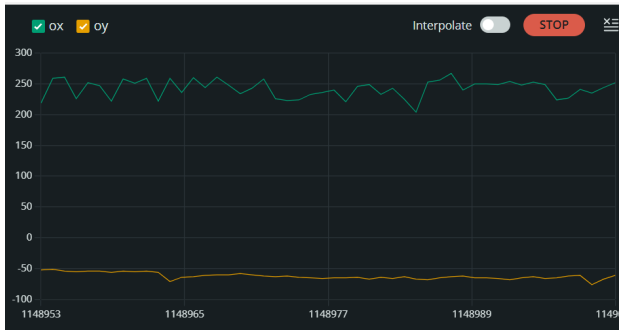


Fig. 4. Serial plot of the x and y readings after the addition of the moving median filter

It mitigated the issue but did not solve the problem. After many attempts to tune the PID control and increase the sample size of the filter, the issue persisted. The team tried another approach. It removed the moving median filter and went back to the Adafruit TouchScreen Library to read the values from the screen. The decision showed there was still noise in the signals regarding the x and y position, but there was less noise and outliers compared to previous methods. The team reverted to using the Adafruit Library to collect the readings, but the exploration into a more efficient program did not come to fruition.

IX. RESISTIVE TOUCHSCREEN IMPROVEMENTS

Using the TouchScreen library to collect data from the touchscreen still had issues with noise, but these were smaller in comparison to other methods that we tried. In the next iteration, a proper filter should be implemented that both reduces noise and takes care of any outliers in the data. Additionally, more data regarding the computation time of each test and the performance of the sensor should be done. This was an oversight the team made when testing. Data should have been collected to discuss the performance of the program. By the time the team realized the performance of the program should have been collected, it was too late as the assembly had progressed, and the testing would have halted the testing for the PID system in the prototype. In future projects, the team will discuss with each other the data the project should discuss and schedule all of the tests in the timeline of the project to prevent this error from occurring again.

X. PROGRAM FUNCTION

The program used in this project is the heart of the embedded system. It makes use of two external libraries, “TouchScreen” to provide an easy way to interface and get output from the resistive touchscreen, and “AccelStepper” to provide easy position, speed, and acceleration control of the stepper motors. The first part of the program defines which analog and digital pins are in use, constants, PID variables, as well as AccelStepper and TouchScreen objects. Upon starting up, the program instructs the platform to smoothly accelerate upward to a predefined position, keeping it level with the surface it is placed on. Once raised up, after two seconds it starts reading the resistive touchpad, trying to detect if the ball is on the platform or not. Once it detects the ball, it makes use of a PID algorithm, with the error defined as the difference between the center of the platform and the position of the ball, in order to calculate two output x and y magnitudes. These values range from -1 to +1 and are fed into the kinematics equations. These calculate the changes of position of each of the three stepper motors. Then, using the predefined maximum speed and acceleration, the steppers are instructed to actuate the platform, the control equations ensuring it attempts to slow the speed of the ball and move it toward the centre of the platform.

XI. MANIPULATOR TROUBLESHOOTING

After the initial assembly, it was noticed that the joints between the legs of each stepper were very stiff. When testing the maneuverability of the platform, these stiff joints caused extra friction, requiring more torque than the steppers could often handle to turn, resulting in the motors skipping. This was due to imperfections in the 3D printed parts, and likely the tolerances of the parts were not well defined or taken into account when printing. The joints were sanded down with a file, and the screw /bolt holding them together was loosened such that the joints rotate smoothly, and the steppers no longer skipped when actuating the platform. Another issue was noticed when the touchpad was mounted on the platform and the ball was placed near a corner. Two of the legs that actuate the platform are connected to the underside near the corners, and when the ball passed near one of these points, the weight seemed to be too much for the servo below, causing it to skip, the platform to tilt improperly, and the ball to roll off. This was fixed programmatically, by simply raising the initial position of the platform higher. This placed more of the force from the weight of the ball down upon the shaft rather than creating a torque around it, lowering the chances of the servo skipping.

XII. PID INTEGRATION AND TUNING

A control system is required to effectively balance the steel ball at the center of the resistive touch screen for precise movement. The error was calculated based on the ball’s position relative to the center of the resistive touch screen to get to a steady-state condition where the ball remains at the center. The initial plan was to implement a Proportional-Integral (PI)

control to handle steady-state errors. This was intended to correct the offsets and ensure that the platform could support the steel ball's weight over time. In theory, PI controllers are good at handling steady-state conditions. However, during testing, it struggled to handle the dynamic behaviour of the system. With the rapid changes in the position of the steel ball on the resistive touch screen, the controller struggled to keep up, resulting in the ball rolling off the platform frequently. Tuning of the proportional and integral coefficients was done but the system was still too slow at reacting to the more rapid movements of the steel ball. The next system that was implemented and tested was a proportional-integral-derivative(PID) controller. To tune it, the proportional term was first adjusted, with the derivative and integral terms set to zero, until the ball was oscillating, but did not often fall off the platform. Then, the integral term was adjusted until the error was minimized somewhat. This did increase the overshoot and intensity of the oscillations, but when the ball was placed close to the center the system performed well so long as it did not move too far from the center. Finally, the derivative term was adjusted until the oscillations were significantly reduced. All three parameters were tweaked over a few days, until finally the values successfully accounted for the dynamics of the ball, and any disturbances were dealt with appropriately.

XIII. RESULTS

The ball can be balanced by the platform successfully. Due to noise from reading the touch pad, the actuation of the platform was often jittery, and this made tuning the PID quite challenging. This issue was initially tackled with moving average and moving median filters, yet these filters did not seem to help very much. The jumpy behaviour of the system was eventually somewhat mitigated by lowering the acceleration value of each stepper, yet it is still not ideal. Another filter, or perhaps more clever programming, could be implemented to account for noise and reject outliers in the signal.

XIV. IMPROVEMENTS FOR NEXT DESIGN

Several improvements can be implemented to enhance the performance and accuracy of the platform in future designs. Using more precise and stronger stepper motors and servos would improve the platform's ability to handle dynamic loads and maintain stability. Additionally, employing stronger materials for the joints would increase the durability and reliability of the mechanical structure. Upgrading to a more sensitive resistive touchscreen could enhance the accuracy of ball position detection, minimizing errors caused by sensor limitations. Implementing a filter, such as a Kalman filter, could further refine data from the touchscreen by compensating for inaccuracies and noise, much like a moving average or median filter.

Moreover, better tuning of the PID controller values would help reduce steady-state error and minimize the jitteriness of the platform, leading to smoother operation. The tolerances and finish of the mechanical parts could also be improved

to facilitate easier assembly and ensure better functional precision. Exploring alternate inverse kinematics equations might enhance positional accuracy, which could further reduce steady-state errors and jitter. Together, these improvements would result in a more robust, accurate, and reliable platform design.

REFERENCES

- [1] J. Schadlbauer, D. R. Walter, and M. L. Husty, "Ammindia," 15th National Conference on Machines and Mechanisms. Available: <http://nacomm2011.ammindia.org/files/papers/121.pdf>, accessed Oct. 7, 2024.
- [2] A. Musa, "I Built a Ball Balancing Robot," YouTube. Available: <https://www.youtube.com/watch?v=v4F-cGDGiEw>, accessed Oct. 6, 2024.
- [3] A. Musa, "Ball Balancing Robot," Instructables. Available: <https://www.instructables.com/Ball-Balancing-Robot/>, accessed Oct. 6, 2024.
- [4] aedmusaPortfolio, "Ball Balancer," Instructables. Available: <https://www.instructables.com/Ball-Balancer/>, accessed Oct. 4, 2024.
- [5] Arduino, "Arduino® MEGA 2560 Rev3." Available: <https://docs.arduino.cc/resources/datasheets/A000067-datasheet.pdf>.
- [6] "DESCRIPTION." Accessed: Nov. 25, 2024. Available: <https://media.digikey.com/pdf/Data>
- [7] 271828-, "3DOF Ball on Plate Using Closed Loop Stepper Motors," Instructables. Available: <https://www.instructables.com/3DOF-Ball-on-Plate-Using-Closed-Loop-Stepper-Motor/>.
- [8] D. Nadeem, W. Cheung, A. Garand, and A. Kotecha, "Ball Balancing Control on a Mobile Stewart Platform," Oct. 6, 2024. Available: <https://docs.google.com/document/d/1FkGIZMT6pJTDde9m-KZ1LLJZfiEGif1r/edit#heading=h.dcn2s53pp19s>.
- [9] "Touch panel for 8.4inch LCD," SHENZHEN VS DISPLAY ELECTRONICS Co., LTD, 2023. Available: <https://www.vslcd.com/8-4inch-4-wire-resistive-touch-screen-vs084tp-a1.html>, accessed Nov. 25, 2024.
- [10] Arduino.cc, 2024. Available: <https://docs.arduino.cc/libraries/adafruit-touchscreen/>, accessed Nov. 25, 2024.
- [11] gmarty2000-ARDUINO, "GitHub - gmarty2000-ARDUINO/arduino-JOYSTICK," GitHub, Mar. 8, 2019. Available: <https://github.com/gmarty2000-ARDUINO/arduino-JOYSTICK>.
- [12] "Analog Joystick — Sensors & Modules," www.electronicwings.com. Available: <https://www.electronicwings.com/sensors-modules/analog-joystick>.
- [13] Arduino.cc, 2024. Available: <https://docs.arduino.cc/built-in-examples/basics/AnalogReadSerial/>.
- [14] K. M. Thakur, "4-Wire Touch Screen Interfacing With Arduino," Steps2Make.com, Instructables. Available: <https://www.instructables.com/4-Wire-Touch-Screen-Interfacing-with-Arduino/>.